جامعة بيرزيت

BIRZEIT UNIVERSITY

Computer Science Department

ARTIFICIAL INTELLIGENCE (COMP338)

Project_2 Report

Classify News Headlines into Categories (Text) by using Decision Tree and Logistic Regression

Student's names	Student's numbers
Sameer Ayman	1221561
Hasan Ahmad Karakra	1220501

Instructor: Dr.AbdelRahman Hamarsha

Section:1

2.1 Data Source:..... 3
2.2 Dataset Characteristics:..... 3
2.3 Dataset Size:..... 3
2.4 Data Sample:..... 3
3. Machine Learning Algorithms Used..... 4
3.1 Logistic Regression..... 4
3.2 Decision Tree..... 4
4. Text Preprocessing and Feature Extraction..... 5
4.1 Text Preprocessing Steps..... 5
4.2 Feature Extraction using TF-IDF..... 5
TF-IDF Configuration:..... 5
5. Model Evaluation Metrics:..... 6
5.1 Accuracy:..... 6
5.2 Precision:..... 6
5.3 Recall:..... 6
5.4 F1-score:..... 6
6. Results and Evaluation..... 7
6.1 Logistic Regression Performance..... 7
6.2 Decision Tree Performance:..... 8
6.3 Model Comparison:..... 9
7. Source Code Explanation:..... 10
7.1 Loading the Dataset:..... 10
7.2 Converting to DataFrame:..... 10
7.3 Text Cleaning Function:..... 10
7.4 Applying Text Cleaning:..... 10
7.5 Feature Extraction with TF-IDF:..... 10
7.6 Training Logistic Regression:..... 11
7.7 Training Decision Tree:..... 11
7.8 Evaluation:..... 11
8. Decision Tree Visualization:..... 12
How It Works:..... 12
Simple Example:..... 12
Why It Performed Worse:..... 12
9. Results Analysis and Discussion:..... 13
9.1 Performance Comparison:..... 13
9.2 Reasons for Logistic Regression Superiority:..... 13
9.3 Class Performance Analysis:..... 13
Conclusion:..... 14

Introduction:

In recent years, automatic text classification has become a fundamental task in the field of Artificial Intelligence and Machine Learning. News classification helps organize large amounts of textual information into predefined categories, making data retrieval and analysis more efficient. In this project, two machine learning algorithms - Logistic Regression and Decision Tree - were applied to classify news headlines into four categories: World, Sports, Business, and Science/Technology. The project focuses on building a complete machine learning pipeline, including text preprocessing, feature extraction using TF-IDF, model training, evaluation, and comparison.

Dataset Overview:

2.1 Data Source:

The AG News dataset from HuggingFace platform was used, which is a widely used benchmark dataset for text classification tasks.

2.2 Dataset Characteristics:

- **Input Feature:**
  - text: News headline (string)
- **Output Label:**
  - label: Category of the news headline
    - 0 → World
    - 1 → Sports
    - 2 → Business
    - 3 → Sci/Tech

2.3 Dataset Size:

- Training set: **120,000** samples
- Test set: **7,600** samples

The dataset is balanced, as each class contains an equal number of samples.

2.4 Data Sample:

Original Text:

French Internet provider Wanadoo will build Dutch broadband network

Label: 3 (Sci/Tech)

3. Machine Learning Algorithms Used

3.1 Logistic Regression

Logistic Regression is a linear classification algorithm used to predict the probability of a sample belonging to a particular class. It works by applying a **sigmoid function** to a linear combination of input features.
Advantages:

- Fast training and prediction
- Works well with high-dimensional data
- Provides probability estimates for classifications

Disadvantages:

- Assumes linear relationship between features and output
- May not perform well with complex nonlinear relationships

3.2 Decision Tree

Decision Tree is a non-linear algorithm that recursively splits data based on feature values. It makes decisions using a series of "if-then" rules until reaching leaf nodes that represent final classifications.
Advantages:

- Easy to understand and interpret
- Doesn't require data normalization
- Can handle both numerical and categorical data

Disadvantages:

- Prone to overfitting
- Sensitive to small changes in data
- Can become very complex with high-dimensional data

4. Text Preprocessing and Feature Extraction

4.1 Text Preprocessing Steps

1. **Convert text to lowercase**: For standardization
2. **Remove special characters**: Using Regular Expressions
3. **Remove extra whitespace**: For clean text

Example:

Before:--> "Wall St. Bears Claw Back Into the Black (Reuters)"

After:--> "wall st bears claw back into the black"

4.2 Feature Extraction using TF-IDF

Machine learning models cannot process text directly; therefore, textual data must be converted into numerical features.

In this project, the **TF-IDF (Term Frequency–Inverse Document Frequency)** Vectorizer was used to transform the cleaned text into numerical feature vectors.

TF-IDF Configuration:

- Maximum number of features: 5000
- N-gram range: (1,1) (unigrams and bigrams)
- English stop words were removed

TF-IDF assigns higher weights to important words while reducing the impact of common words, making it suitable for text classification tasks.

Machine Learning Models:

5. Model Evaluation Metrics:

5.1 Accuracy:

Accuracy = (Number of Correct Predictions) / (Total Number of Predictions)

===>Measures the overall proportion of correct predictions.

5.2 Precision:

Precision = TP / (TP + FP)

Where:

- TP (True Positive): Correct positive predictions
- FP (False Positive): Incorrect positive predictions

Measures: "Of all instances predicted as class X, how many were actually class X?"

5.3 Recall:

Recall = TP / (TP + FN)

Where FN (False Negative): Incorrect negative predictions

Measures: "Of all actual instances of class X, how many were correctly retrieved?"

5.4 F1-score:

F1 = 2 * (Precision * Recall) / (Precision + Recall)

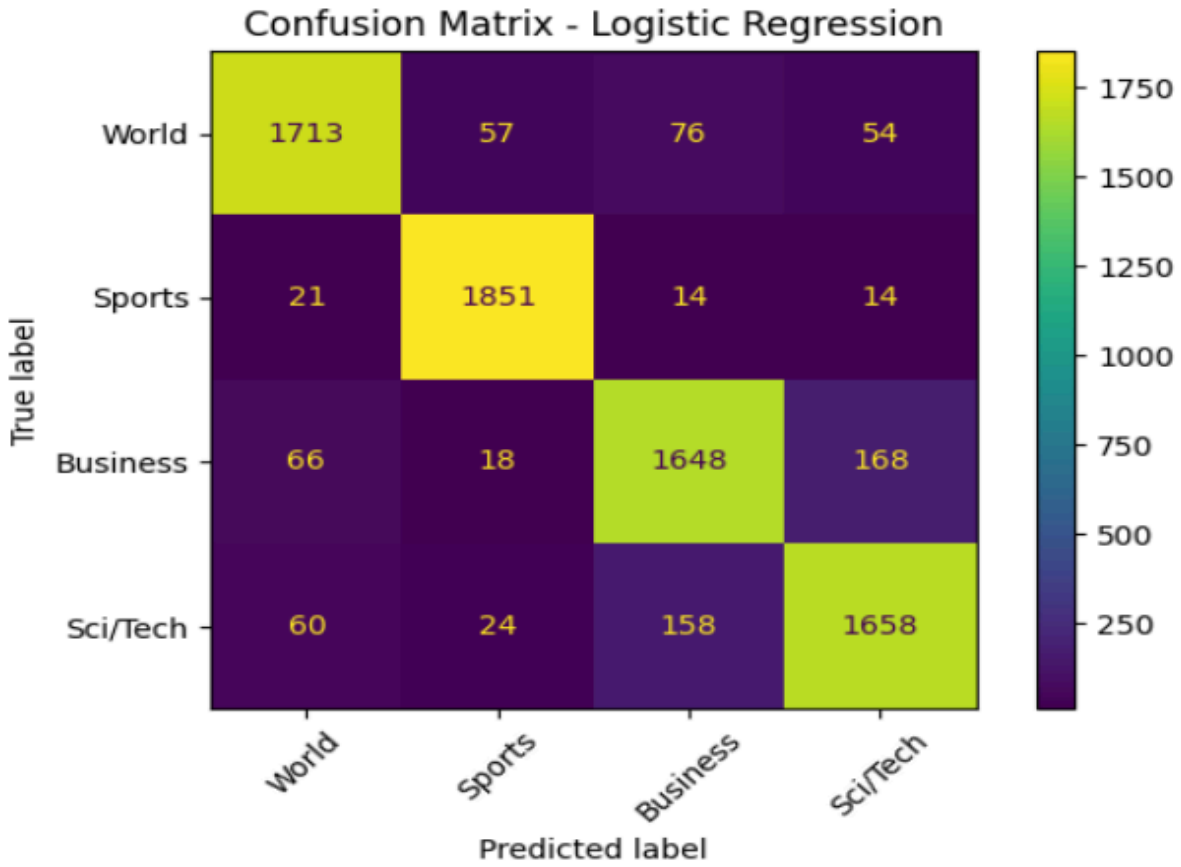
Harmonic mean of Precision and Recall, giving equal weight to both metrics.

6. Results and Evaluation

6.1 Logistic Regression Performance

===> Insert Logistic Regression Confusion Matrix Image Here

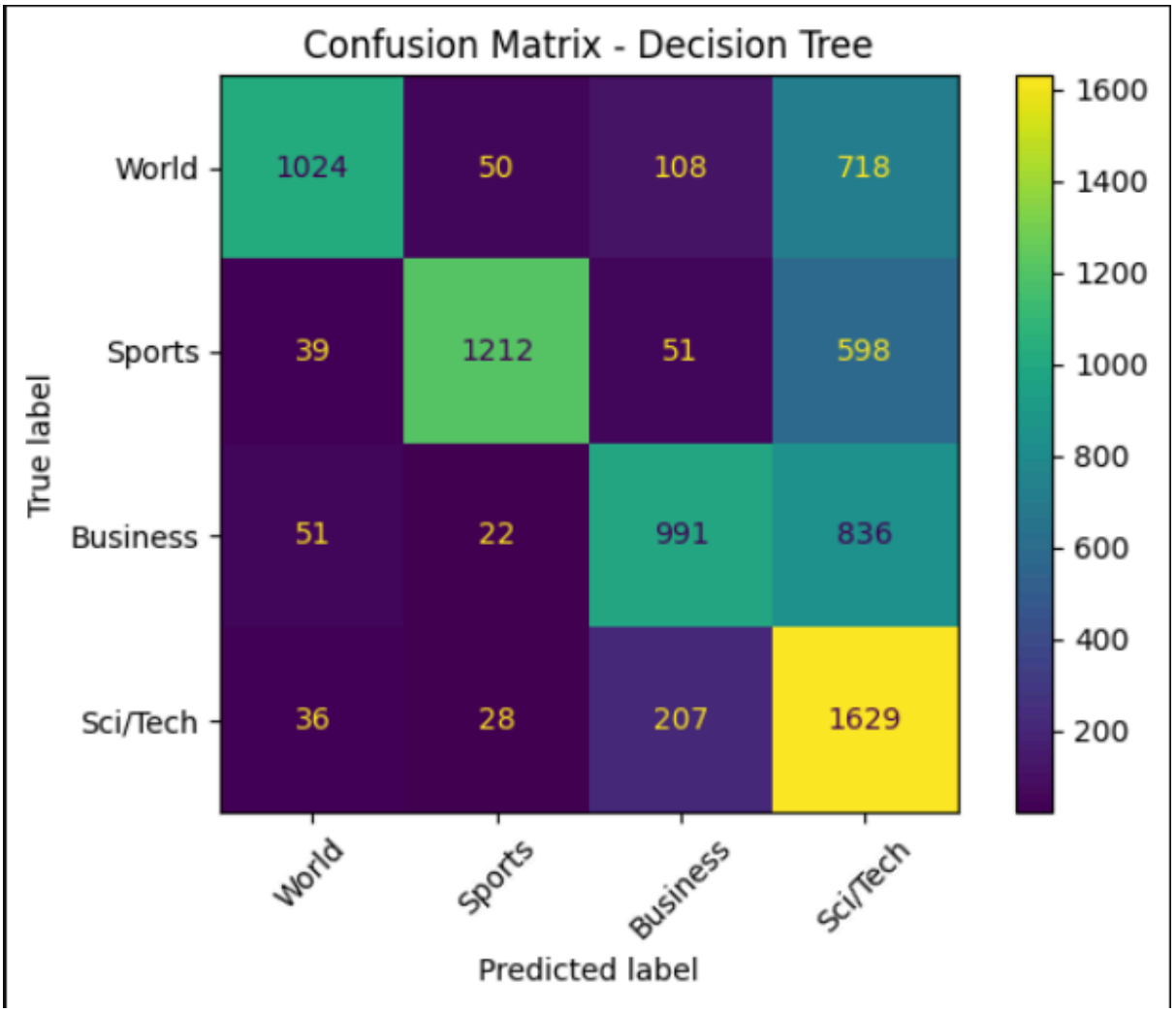
```
Accuracy: 0.9039
Classification Report for logistec  regresion :
              precision    recall  f1-score   support

       World       0.92      0.90      0.91      1900
      Sports       0.95      0.97      0.96      1900
    Business       0.87      0.87      0.87      1900
    Sci/Tech       0.88      0.87      0.87      1900

    accuracy                           0.90      7600
   macro avg       0.90      0.90      0.90      7600
weighted avg       0.90      0.90      0.90      7600
```



6.2 Decision Tree Performance:

===> Insert Decision Tree Confusion Matrix Image Here

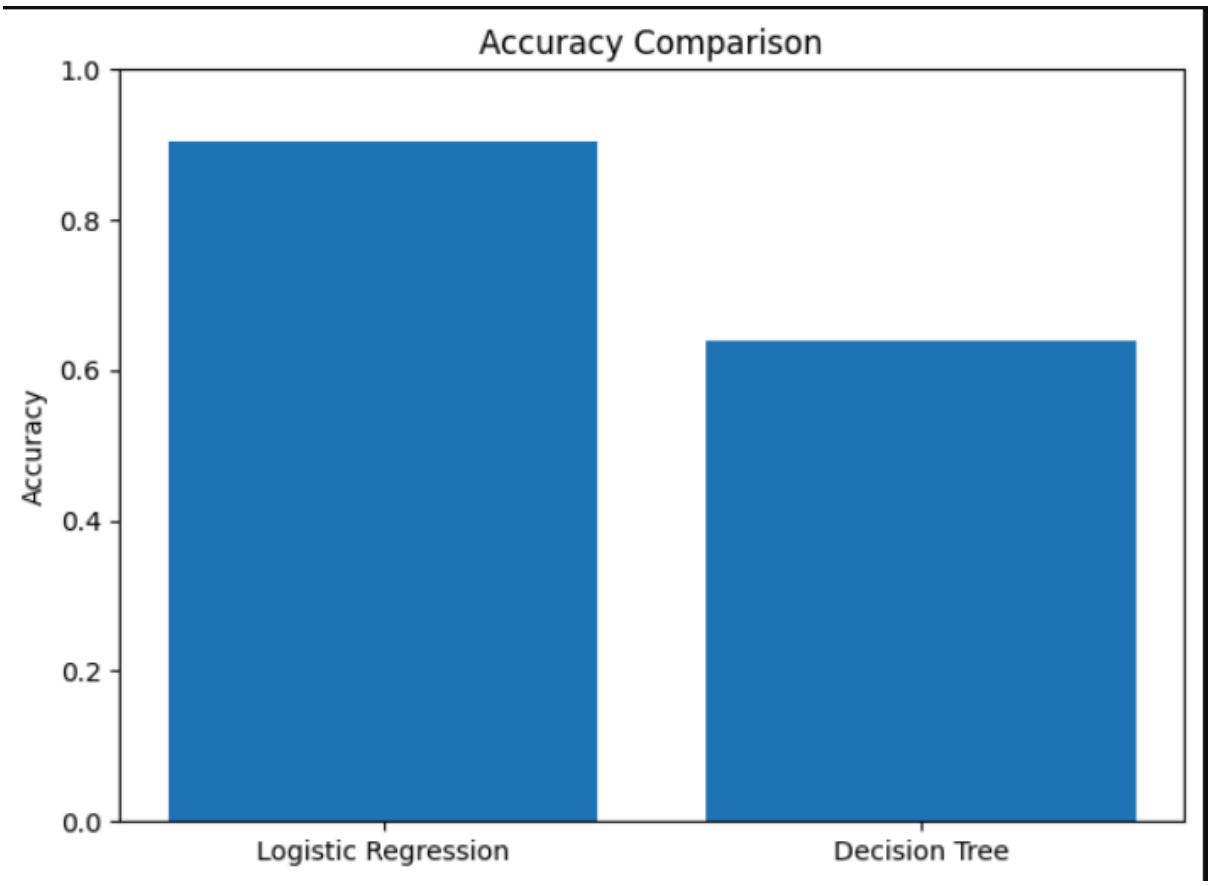
```
Decision Tree Accuracy: 0.6389
Decision Tree Classification Report:
              precision    recall  f1-score   support

       World       0.89      0.54      0.67      1900
      Sports       0.92      0.64      0.75      1900
    Business       0.73      0.52      0.61      1900
    Sci/Tech       0.43      0.86      0.57      1900

    accuracy                           0.64      7600
   macro avg       0.74      0.64      0.65      7600
weighted avg       0.74      0.64      0.65      7600
```



6.3 Model Comparison:

===> Insert Accuracy Comparison Bar Chart Image Here

Metric	Logistic Regression	Decision Tree
Accuracy	0.9039	0.6389
Precision (Average)	0.9042	0.7425
Recall (Average)	0.9039	0.6389
F1-score (Average)	0.9039	0.6500



7. Source Code Explanation:

7.1 Loading the Dataset:

```
from datasets import load_dataset
ds = load_dataset("wangrongsheng/ag_news")
```

Loads the dataset from HuggingFace using the datasets library.

7.2 Converting to DataFrame:

```
train_df = pd.DataFrame(ds['train'])
test_df = pd.DataFrame(ds['test'])
```

Converts data to Pandas DataFrames for easier manipulation.

7.3 Text Cleaning Function:

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    text = re.sub(r'\s+', ' ', text)
    return text
```

The function performs:

1. Convert text to lowercase
2. Remove all non-alphabetical characters
3. Remove extra whitespace

7.4 Applying Text Cleaning:

```
train_df['after_clean'] = train_df['text'].apply(clean_text)
test_df['after_clean'] = test_df['text'].apply(clean_text)
```

7.5 Feature Extraction with TF-IDF:

```
tfidf = TfidfVectorizer(stop_words='english', max_features=5000)
X_train = tfidf.fit_transform(train_df['after_clean'])
X_test = tfidf.transform(test_df['after_clean'])
```

7.6 Training Logistic Regression:

```
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

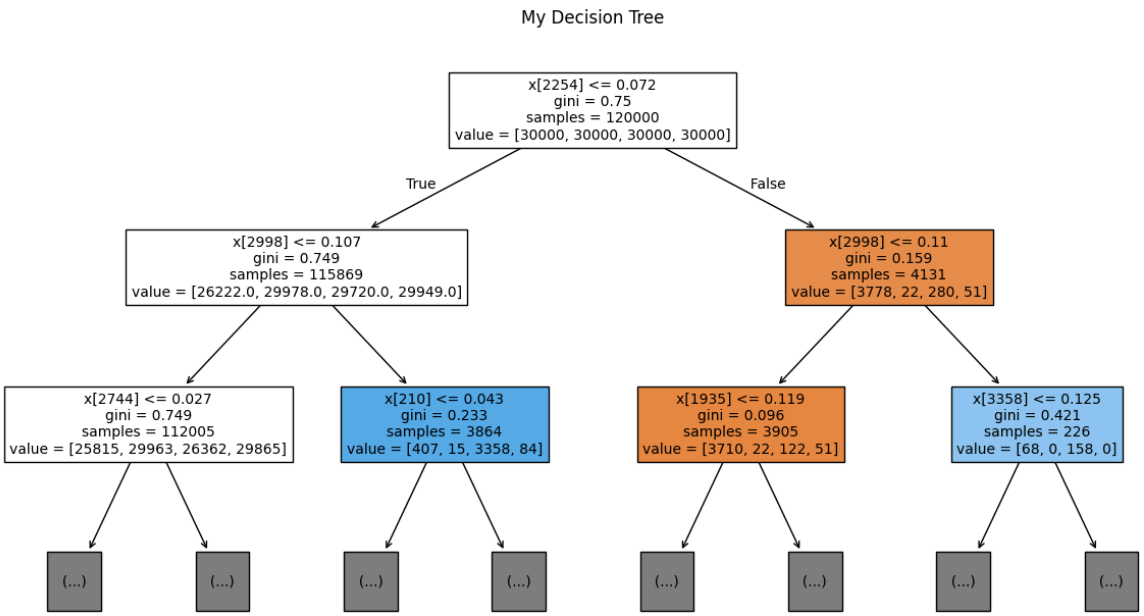
7.7 Training Decision Tree:

```
dt_model = DecisionTreeClassifier(
    random_state=42,
    max_depth=40,
    min_samples_split=10,
    min_samples_leaf=5
)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
```

7.8 Evaluation:

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
print(classification_report(y_test, y_pred))
```

8. Decision Tree Visualization:



The decision tree makes classification decisions through simple if-then rules:

How It Works:

- First check: Does the headline contain word #2254?
- If YES: Check word #2998
- If NO: Check different words
- Continue for up to 40 checks
- Final step: Choose category (World, Sports, Business, or Sci/Tech)

Simple Example:

1. Is "football" in the headline? → Yes → SPORTS
2. Is "stock" in the headline? → Yes → BUSINESS
3. Is "science" in the headline? → Yes → SCI/TECH
4. Otherwise → WORLD

Why It Performed Worse:

- **Too many rules (40 levels deep)**
- **Learns specific examples instead of general patterns**
- **Confused by too many word features (5000+ words)**

9. Results Analysis and Discussion:

===> discuss the difference in performance measurements between Logistic Regression and Decision Tree models

9.1 Performance Comparison:

Logistic Regression significantly outperformed Decision Tree across all evaluation metrics. The accuracy difference was approximately 26.5%, indicating that Logistic Regression is more suitable for this task.

9.2 Reasons for Logistic Regression Superiority:

1. Nature of Text Data:
    - Text data after TF-IDF is high-dimensional and sparse
    - Logistic Regression handles sparse data efficiently
    - Decision Trees struggle with high-dimensional data
2. Overfitting in Decision Tree:
    - Despite setting max_depth=40, the tree remains complex
    - Learning from noise in text data
3. Feature Distribution:
    - Logistic Regression works better with linear features
    - Decision Trees need features with clear classification impact

9.3 Class Performance Analysis:

In Logistic Regression:

- Best Performance: Sports (F1-score: 0.96) due to distinctive terminology
- Lowest Performance: Business and Science (F1-score: 0.87) due to overlapping terms

In Decision Tree:

- Clear Contradiction: High precision in World (0.89) but low recall (0.54)
- Problem with Sci/Tech: Very low precision (0.43) due to misclassifications

Conclusion:

In conclusion, this project demonstrates that Logistic Regression outperforms Decision Tree for text classification of news headlines using TF-IDF features, achieving 90.39% accuracy versus 63.89%. The results highlight the importance of selecting appropriate algorithms for text data and implementing proper preprocessing techniques. Future work could explore advanced algorithms like Random Forest or Neural Networks, along with improved text processing methods for even better performance.